

# गिनती समझाना

## 1. Big Picture

The problem is **LeetCode 357 — Count Numbers with Unique Digits**.

The goal is:

Given an integer  $n$ , count how many numbers from  $0$  to  $10^n - 1$  have **no repeated digit**.

The key idea in the code is **counting permutations**, not actually generating every number.

The transcript's main teaching goal is to understand the counting process and then implement it efficiently.

For example:

- $n = 0 \rightarrow$  only  $0 \rightarrow$  answer  $1$
- $n = 1 \rightarrow 0, 1, 2, \dots, 9 \rightarrow$  answer  $10$
- $n = 2 \rightarrow$  numbers from  $0$  to  $99$  having unique digits  $\rightarrow$  answer  $91$
- $n = 3 \rightarrow$  answer  $739$

The important insight is:

**Once a digit is used, you cannot use it again.**

So the number of choices decreases at every position.

## 2. Prerequisites

Before understanding the code, you need only three ideas.

### 2.1 Unique digits

A number has unique digits if no digit occurs twice.

Examples:

```
text
123      → unique
507      → unique
9081     → unique
```

But:

```
text
121      → not unique
100      → not unique
455      → not unique
```

### 2.2 Counting choices

Suppose you want to create a 2-digit number.

For the first digit:

```
text
1, 2, 3, 4, 5, 6, 7, 8, 9
```

There are **9 choices**.

Why not  $0$ ?

Because a normal 2-digit number cannot start with  $0$ .

After selecting the first digit, there are:

```
text
10 - 1 = 9
```

remaining digits for the second position.

So:

```
text
9 × 9 = 81
```

valid 2-digit numbers.

### 2.3 Multiplication rule

If one position has **A** choices and another position has **B** choices, then:

```
text
Total possibilities = A × B
```

For example:

```
text

first digit → 9 choices
second digit → 9 choices

total = 9 × 9 = 81
```

This is the mathematical foundation of the entire solution.

---

### 3. Detailed Explanation

#### 3.1 Understanding the actual problem

We need to count numbers satisfying:

1. Number has at most  $n$  digits.
2. Every digit must be different.
3. We count numbers from 0 through  $10^n - 1$ .

Suppose:

```
text

n = 2
```

Then we consider:

```
text

0 → 99
```

The valid numbers include:

```
text

0
1
2
...
9
10
12
13
...
98
```

But not:

```
text

11
22
33
...
99
```

because these contain repeated digits.

---

#### 3.2 The most important observation

Instead of checking every number individually, count numbers based on their **number of digits**.

For  $n = 2$ , we count:

```
text

0-digit/zero case
+
1-digit numbers
+
2-digit numbers
```

For  $n = 3$ :

```
text
0
+
1-digit numbers
+
2-digit numbers
+
3-digit numbers
```

This is exactly what the code's `ans` variable is doing.

### 3.3 Counting 1-digit numbers

There are exactly:

```
text
0,1,2,3,4,5,6,7,8,9
```

So:

```
text
10
```

numbers.

Notice that the code starts with:

```
C++
int ans = 10;
```

Why?

Because all 10 single-digit numbers are automatically unique.

There is no possibility of repetition when there is only one digit.

### 3.4 Counting 2-digit numbers

Now consider:

```
text
12
```

We need two different digits.

#### First digit

It cannot be zero.

Therefore:

```
text
1 → 9
```

There are:

```
text
9 choices
```

#### Second digit

We have 10 possible digits:

```
text
0,1,2,3,4,5,6,7,8,9
```

But one digit has already been used.

Therefore:

```
text
10 - 1 = 9
```

choices remain.

So:

```
text
9 × 9 = 81
```

two-digit numbers.

Therefore, for  $n = 2$ :

```
text
1-digit numbers = 10
2-digit numbers = 81

answer = 10 + 81
        = 91
```

### 3.5 Counting 3-digit numbers

Now let's create:

```
text
ABC
```

where:

```
text
A ≠ B
A ≠ C
B ≠ C
```

#### First digit

Cannot be zero:

```
text
9 choices
```

#### Second digit

One digit is already used:

```
text
9 choices
```

#### Third digit

Two digits are already used:

```
text
8 choices
```

Therefore:

```
text
9 × 9 × 8
= 648
```

three-digit numbers have unique digits.

So for  $n = 3$ :

```
text
1-digit = 10
2-digit = 81
3-digit = 648

total = 10 + 81 + 648
      = 739
```

Therefore:

```
text
n = 3
answer = 739
```

### 3.6 General pattern

Look at the pattern:

| Number of digits | Calculation | Count |
|------------------|-------------|-------|
| 1                | 10          | 10    |
| 2                | 9 × 9       | 81    |
| 3                | 9 × 9 × 8   | 648   |

| Number of digits | Calculation                                               | Count   |
|------------------|-----------------------------------------------------------|---------|
| 4                | $9 \times 9 \times 8 \times 7$                            | 4536    |
| 5                | $9 \times 9 \times 8 \times 7 \times 6$                   | 27216   |
| 6                | $9 \times 9 \times 8 \times 7 \times 6 \times 5$          | 136080  |
| 7                | $9 \times 9 \times 8 \times 7 \times 6 \times 5 \times 4$ | 544320  |
| 8                | $\dots \times 3$                                          | 1095920 |
| 9                | $\dots \times 2$                                          | 1569120 |
| 10               | $\dots \times 1$                                          | 1569120 |

The pattern is:

text

```

first position → 9 choices
second        → 9 choices
third         → 8 choices
fourth        → 7 choices
fifth         → 6 choices
...

```

The first position is special because 0 cannot be placed there.

## 3.7 Now understand the code

The cleaned-up code is:

C++

```

class Solution {
public:
    int countNumbersWithUniqueDigits(int n) {
        if (n == 0) return 1;

        int ans = 10;
        int count = 9;
        int available = 9;

        for (int digits = 2; digits <= n; digits++) {
            count *= available;
            ans += count;
            available--;
        }

        return ans;
    }
};

```

Now let's understand why every variable exists.

### 3.8 if (n == 0) return 1

C++

```

if (n == 0) return 1;

```

This is an important edge case.

The range becomes:

text

```

0 <= x < 10^0

```

Since:

text

```

10^0 = 1

```

we have:

text

```

0 <= x < 1

```

Only:

text

```

0

```

exists.

Therefore:

```
text
answer = 1
```

Without this condition, the code would initialize:

```
C++
ans = 10;
```

which would be incorrect for  $n = 0$ .

---

### 3.9 ans

```
C++
int ans = 10;
```

ans means:

Total number of valid numbers found so far.

Initially, we already know that there are 10 valid one-digit numbers:

```
text
0,1,2,3,4,5,6,7,8,9
```

Therefore:

```
text
ans = 10
```

Think of `ans` as our **running total**.

---

### 3.10 count

```
C++
int count = 9;
```

This represents the number of valid numbers for the **current digit length**.

Initially we're preparing to calculate 2-digit numbers.

For a 2-digit number:

```
text
first digit = 9 choices
```

So:

```
text
count = 9
```

Then the loop multiplies it by another 9.

---

### 3.11 available

```
C++
int available = 9;
```

This tells us:

How many unused digits are available for the next position?

For a 2-digit number:

```
text
first digit → 9 choices
second digit → 9 choices
```

So initially:

```
text
available = 9
```

After processing the second digit, one more digit becomes unavailable.

Therefore:

```
C++
available--;
```

makes:

```
text
9 → 8
```

Then for a 3-digit number, we multiply by 8.

---

### 3.12 The `for` loop

```
C++
for (int digits = 2; digits <= n; digits++)
```

Why start from 2?

Because we have already handled the 1-digit numbers:

```
C++
ans = 10;
```

So now we only need:

```
text
2 digits
3 digits
4 digits
...
n digits
```

### 3.13 First loop iteration

Suppose:

```
text
n = 3
```

Initially:

```
text
ans = 10
count = 9
available = 9
```

`digits = 2`

Execute:

```
C++
count *= available;
```

Therefore:

```
text
count = 9 × 9
      = 81
```

So there are:

```
text
81
```

valid 2-digit numbers.

Then:

```
C++
ans += count;
```

becomes:

```
text
ans = 10 + 81
     = 91
```

Then:

```
C++
available--;
```

becomes:

```
text
available = 8
```

---

### 3.14 Second loop iteration

Now:

```
text
digits = 3
```

Current state:

```
text
ans = 91
count = 81
available = 8
```

Execute:

```
C++
count *= available;
```

Therefore:

```
text
count = 81 × 8
       = 648
```

That's exactly the number of valid 3-digit numbers.

Then:

```
C++
ans += count;
```

gives:

```
text
ans = 91 + 648
     = 739
```

Then:

```
C++
available--;
```

gives:

```
text
available = 7
```

The loop ends because:

```
text
digits = 4
```

is greater than:

```
text
n = 3
```

Finally:

```
C++
return ans;
```

returns:

```
text
739
```

---

### 3.15 Complete dry run for `n = 5`

Let's trace the entire algorithm.

Initial:

```
text
ans = 10
count = 9
available = 9
```

#### 2 digits

```
text
count = 9 × 9
      = 81

ans = 10 + 81
    = 91

available = 8
```

#### 3 digits

```
text
count = 81 × 8
      = 648

ans = 91 + 648
    = 739

available = 7
```

#### 4 digits

```
text
count = 648 × 7
      = 4536

ans = 739 + 4536
    = 5275

available = 6
```

#### 5 digits

```
text
count = 4536 × 6
      = 27216

ans = 5275 + 27216
    = 32491
```

Therefore:

```
text
n = 5
answer = 32491
```

### 3.16 The code's mathematical idea

The code is essentially calculating:

```
text
10
+
9 × 9
+
9 × 9 × 8
+
9 × 9 × 8 × 7
+
...
```

For example, when `n = 4`:

```
text
answer
= 10
+ (9 × 9)
+ (9 × 9 × 8)
+ (9 × 9 × 8 × 7)

= 10 + 81 + 648 + 4536

= 5275
```

That is the entire mathematical logic behind the solution.

### 3.17 Why does available start at 9?

This is one of the most important details.

You might initially think:

```
There are 10 digits, so why not start available = 10?
```

Because the first digit has already consumed one digit, and we are calculating the next position.

For example:

```
text
First digit = 5
```

Now the second digit cannot be:

```
text
5
```

So only:

```
text
0,1,2,3,4,6,7,8,9
```

remain.

That's exactly:

```
text
9 choices
```

So:

```
C++
available = 9;
```

is correct.

### 3.18 Why does available-- happen?

Suppose we're constructing:

```
text
1234
```

After choosing:

```
text
1
```

there are:

```
text
9
```

choices remaining.

After choosing:

```
text
2
```

there are:

```
text
```

8

choices remaining.

After choosing:

```
text
```

3

there are:

```
text
```

7

choices remaining.

So:

```
text
```

9 → 8 → 7 → 6 → ...

That's exactly what:

```
C++
```

```
available--;
```

does.

### 3.19 Why do we multiply `count`?

This line is the heart of the solution:

```
C++
```

```
count *= available;
```

Suppose:

```
text
```

```
count = 81  
available = 8
```

81 represents all valid ways of creating the first 2 positions.

Now we want to add another digit.

Each of those 81 possibilities can choose from 8 unused digits.

Therefore:

```
text
```

81 × 8 = 648

This is the multiplication rule for independent sequential choices.

### 3.20 Why don't we generate the actual numbers?

A brute-force approach would be:

```
text
```

```
0  
1  
2  
3  
...  
999999999
```

and check every number.

That would be extremely inefficient.

The counting approach asks a much better question:

"How many choices are available at each position?"

We don't care about the actual numbers.

We only care about the number of possible choices.

This changes the problem from:

```
text
Generate millions/billions of numbers
```

to:

```
text
Perform at most 9 multiplications
```

That's the major algorithmic insight.

### 3.21 Important limitation: maximum useful $n$

There are only 10 unique digits:

```
text
0 1 2 3 4 5 6 7 8 9
```

Therefore, you cannot construct a number with more than 10 unique digits.

For example:

```
text
1234567890
```

has 10 unique digits.

But a hypothetical 11-digit number would require repeating at least one digit.

Therefore, after 10 digits, the number of additional unique-digit numbers is:

```
text
0
```

The standard LeetCode constraints make this manageable, and the loop effectively has no useful work beyond the available digit choices.

### 3.22 Complexity

There is a very important difference between **brute force** and this approach.

#### Brute force

You could potentially inspect up to:

```
text
10^n
```

numbers.

So the complexity grows exponentially with  $n$ .

#### This solution

The loop runs at most around 10 times.

Therefore:

```
text
Time Complexity: O(n)
```

and because only a few integer variables are used:

```
text
Space Complexity: O(1)
```

This is dramatically better than generating all numbers.

## 4. Key Takeaways

The five things you should remember are:

### 1. Start with 10

```
C++
ans = 10;
```

because:

```
text
0 through 9
```

are all valid one-digit numbers.

## 2. First digit has 9 choices

A multi-digit number cannot start with 0.

Therefore:

```
text
9 choices
```

for the first digit.

## 3. Every next position loses one choice

```
text
9 → 8 → 7 → 6 → ...
```

because digits cannot repeat.

## 4. Multiply choices

For example:

```
text
3-digit:

9 × 9 × 8 = 648
```

## 5. Add counts for every digit length

For  $n = 3$ :

```
text
10
+ 81
+ 648
= 739
```

The code is simply automating this mathematical pattern.

## 5. Common Mistakes / Misunderstandings

### Mistake 1: Starting with 10 choices for the first digit

Wrong:

```
text
10 × 9 × 8
```

because that includes numbers such as:

```
text
012
```

which is not a 3-digit number.

Correct:

```
text
9 × 9 × 8
```

### Mistake 2: Forgetting the single-digit numbers

If  $n = 2$ , don't calculate only:

```
text
9 × 9 = 81
```

The answer includes:

```
text
0 through 9
```

as well.

Therefore:

```
text
81 + 10 = 91
```

### Mistake 3: Counting only exactly $n$ digits

The problem asks for numbers less than  $10^n$ , not numbers having exactly  $n$  digits.

So for:

```
text
n = 3
```

we count:

```
text
1-digit
+
2-digit
+
3-digit
```

not just 3-digit numbers.

### Mistake 4: Thinking `count` means the total answer

It doesn't.

```
C++
count
```

means:

```
Number of valid numbers having the current number of digits.
```

Whereas:

```
C++
ans
```

means:

```
Total valid numbers across all digit lengths processed so far.
```

### Mistake 5: Thinking this is DP

This solution is often grouped with counting/combinatorics rather than traditional DP.

There is a **state-like progression**:

```
text
9 choices
↓
8 choices
↓
7 choices
↓
...
```

but there is no need for a DP table because the state is extremely simple and can be maintained with two variables.

## 6. Quick Revision

Memorize this pattern:

```
text
1 digit:
10

2 digits:
9 × 9

3 digits:
9 × 9 × 8

4 digits:
9 × 9 × 8 × 7

5 digits:
9 × 9 × 8 × 7 × 6
```

So the general formula is:

```
text

answer =
10
+ 9×9
+ 9×9×8
+ 9×9×8×7
+ ...
```

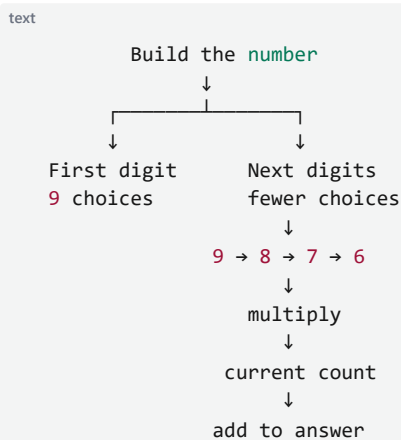
And the implementation:

```
C++

int ans = 10;
int count = 9;
int available = 9;

for (int digits = 2; digits <= n; digits++) {
    count *= available;
    ans += count;
    available--;
}
```

Mental model:



## 7. Questions You Should Be Able to Answer

Before considering this problem fully understood, you should be able to answer these without looking at the code:

1. Why are there 10 valid numbers when `n = 1`?
2. Why does the first digit have only 9 choices?
3. Why does the second digit have 9 choices instead of 10?
4. Why does the third digit have 8 choices?
5. Why do we add the counts for 1-digit, 2-digit, 3-digit, etc.?
6. What exactly does `count` represent?
7. What exactly does `available` represent?
8. Why does `available--` happen after every iteration?
9. Why do we use multiplication rather than addition for the choices?
10. Why does `n == 0` return 1?
11. Why isn't brute force a good solution?
12. What is the time complexity?
13. What is the space complexity?
14. For `n = 3`, why is the answer 739?
15. What happens when we run out of unique digits?

The one-line intuition to remember

Count how many choices each digit position has, multiply those choices to get the count for that digit length, then add the counts for all lengths up to `n`.